

Informe Preliminar

Pipeline de Streaming en Tiempo Real
para Detección de Anomalías en Criptomonedas

Arquitectura de Grandes Volúmenes de Datos

ITAM - Primavera 2026

Prof. Wilmer Pereira

Adolfo Yunes

adolfoyunes1@gmail.com

Fecha: 22 de abril de 2026

1. Introducción

El presente informe describe el diseño e implementación de un pipeline de procesamiento de datos en tiempo real para la detección de anomalías en mercados de criptomonedas. El proyecto se desarrolla como trabajo final de la materia Arquitectura de Grandes Volúmenes de Datos en el ITAM.

El objetivo central del proyecto es la comparación de la ejecución de una misma aplicación de streaming en dos arquitecturas de hardware distintas: una máquina macOS con Apple M1 (sin GPU) y una máquina Linux con GPU NVIDIA. El foco está en medir y comparar métricas de rendimiento (tiempo de ejecución, shuffle, GC, spill, etc.), no en la complejidad de los modelos de machine learning.

La fuente de datos es el WebSocket público de Binance, que provee trades reales de criptomonedas en tiempo real sin requerir autenticación. Se monitorean 8 pares contra USDT: BTC, ETH, SOL, BNB, XRP, DOGE, ADA y AVAX.

2. Stack Tecnológico

Componente	Versión	Notas
Python	3.11.7	Anaconda
PySpark	3.5.4	Binario local + pip
Scala	2.12	Conector Kafka
Java	17	OpenJDK (Homebrew)
Apache Kafka	4.2.0	KRaft mode (sin Zookeeper)
Hadoop	3.3.6	Solo libs, sin HDFS activo
Tableau	Estudiantil	Visualización (Fase 6)

Configuración clave de Spark

- `spark.hadoop.fs.defaultFS = file:///` - se fuerza filesystem local para evitar errores de conexión a HDFS (que no está activo).
- `spark.sql.shuffle.partitions = 4` - reducido del default 200 para modo local con un solo nodo.
- `spark.driver.memory = 2g` - ajustado a los 8 GB de RAM total de la Máquina A.
- `spark.jars.packages` - el conector Kafka se descarga automáticamente de Maven.

3. Arquitectura General del Pipeline

El pipeline se ejecuta en tres momentos secuenciales (ninguna fase comienza hasta que la anterior ha terminado completamente):

Momento 1 - Primer pase de streaming: ingesta de datos desde Binance vía Kafka, cálculo de estadísticos y features por ventana de tiempo, acumulación de datos, y entrenamiento de K-Means en batch para detección de anomalías no supervisada.

Momento 2 - Entrenamiento supervisado (batch): se entrena un modelo de Logistic Regression usando los datos etiquetados por K-Means como ground truth.

Momento 3 - Segundo pase de streaming: inferencia en tiempo real con el modelo supervisado entrenado, clasificando nuevas ventanas de datos como normales o anómalas.

Diagrama de flujo



4. Descripción Detallada de las Fases

Fase 0 - Diagnóstico del entorno

Antes de ejecutar el pipeline, se verifica la correcta instalación de todos los componentes: Spark, Kafka, Java, y la conectividad entre PySpark y Kafka. Se ejecuta un script `hardware_info.py` que captura las especificaciones del hardware (CPU, RAM, GPU si existe, disco) y las exporta a JSON. Este paso se repite en ambas máquinas para la tabla comparativa del informe final.

Adicionalmente, `verify_environment.py` prueba la cadena completa: crea un topic de prueba en Kafka, produce y consume un mensaje, y verifica que PySpark puede leer de Kafka correctamente.

Fase 1 - Productor de datos (Binance -> Kafka)

El script `producer/binance_producer.py` se conecta al WebSocket público de Binance y recibe trades en tiempo real de 8 pares de criptomonedas. Cada trade se parsea y publica como un mensaje JSON en el topic 'crypto-trades' de Kafka.

El esquema de cada mensaje es:

```
{
  "symbol": "BTCUSDT",
  "price": 67543.21,
  "quantity": 0.0023,
  "timestamp": 1714000000000,
  "is_buyer_maker": true
}
```

El productor implementa reconexión automática (Binance desconecta cada 24 horas), manejo de señales para cierre limpio con Ctrl+C, y reportes de throughput cada 500 mensajes. La tasa observada es de aproximadamente 50-200 mensajes por segundo dependiendo de la actividad del mercado.

Fase 2 - Primer streaming: Estadísticos + K-Means

Esta es la fase más compleja del pipeline. Spark Structured Streaming lee del topic de Kafka y ejecuta tres etapas dentro de cada micro-batch:

Parte A - Estadísticos por ventana de tiempo

Se agrupan los trades por símbolo y ventana de tiempo (1 minuto de duración, 30 segundos de slide) y se calculan: mínimo, máximo, promedio y varianza del precio; conteo de trades; y volumen total. Se usa un watermark de 30 segundos para manejar datos tardíos.

Parte B - Feature engineering

Se calculan 4 features por ventana para alimentar los modelos de ML:

Feature	Descripción	Intuición
pct_return	Retorno % del precio	Dirección y magnitud del movimiento
volatility	Stddev / precio promedio	Fluctuación relativa al nivel
volume_intensity	Volumen / num. trades	Tamaño promedio de transacción
price_speed	Rango / precio promedio	Amplitud relativa del rango

Parte C - K-Means (no supervisado)

Una vez acumulados suficientes datos (controlado por ACCUMULATION_SECONDS), se detiene el streaming y se entrena K-Means en modo batch sobre todos los features acumulados. K-Means agrupa los datos en k clusters minimizando la distancia al cuadrado de cada punto a su centroide.

Se prueban $k = 3, 4$ y 5 , seleccionando el que maximice el Silhouette Score. Luego se calcula la distancia euclidiana de cada punto a su centroide asignado; los puntos con distancia superior al percentil 95 se etiquetan como 'anómalo' (el otro 95% como 'normal'). Los clusters representan estados del mercado: calma, volatilidad moderada, y movimiento extremo.

El pipeline incluye VectorAssembler (combina las 4 features en un vector) y StandardScaler (normaliza a media 0, std 1) antes de K-Means, para que ningún feature domine por su escala.

Fase 3 - Entrenamiento supervisado (Logistic Regression)

Se lee el dataset etiquetado por K-Means y se entrena un modelo de Logistic Regression para clasificación binaria: 0 = normal, 1 = anómalo. Se eligió Logistic Regression por ser el clasificador más simple de Spark ML, dado que el foco del proyecto es la comparación de arquitecturas, no la complejidad del modelo.

El pipeline de ML encadena tres etapas: VectorAssembler -> StandardScaler -> LogisticRegression. La regularización es L2 pura (Ridge) con regParam = 0.1. Se hace split 80/20 con semilla fija (42) para reproducibilidad.

Las métricas de evaluación incluyen:

- Accuracy: proporción de predicciones correctas.
- Precision (weighted): de los predichos como anómalos, cuántos lo eran.
- Recall (weighted): de los anómalos reales, cuántos se detectaron.
- F1-score (weighted): media armónica de precision y recall.
- AUC-ROC: capacidad de discriminación independiente del umbral.

Fase 4 - Segundo streaming: Inferencia en tiempo real

Se reactiva Spark Structured Streaming leyendo del mismo topic de Kafka, pero con startingOffsets = 'latest' para procesar solo datos nuevos. Se calculan exactamente las mismas features que en la Fase 2 (reutilizando las funciones compute_window_statistics y compute_features) y se aplica el modelo de Logistic Regression cargado desde disco.

Cada ventana de tiempo se clasifica como normal o anómala, y las predicciones se escriben a Parquet con la columna de probabilidad para análisis de confianza.

Fase 5 - Captura de métricas de Spark UI

Mientras corre la Fase 2 o 4, se ejecuta capture_metrics.py en paralelo. Este script se conecta a la API REST de Spark UI (<http://localhost:4040/api/v1/>) y descarga métricas de jobs, stages, executors, environment y streaming queries.

Las métricas clave que se capturan para la comparación son:

- Tiempo de ejecución de los jobs
- Executor Run Time y Scheduler Delay
- GC Time (garbage collection)
- Shuffle Read/Write (bytes transferidos entre stages)
- Spill a memoria y disco
- Tasa de streaming (input rate, processing rate)
- Configuración del entorno

Fase 6 - Visualización con Tableau

Se consolidan los outputs de estadísticos, predicciones y métricas en archivos CSV consumibles por Tableau. Se preparan datos para dashboards animados que muestren la serie temporal de precios con marcas de anomalía, estadísticos por ventana, y la comparación de métricas de rendimiento entre ambas máquinas.

5. Arquitecturas de Hardware a Comparar

Característica	Máquina A (Mac)	Máquina B (Linux)
Sistema Operativo	macOS (Darwin)	Linux
CPU	Apple M1	x86_64 (por confirmar)
Cores	8	Por confirmar
RAM	8 GB	16 GB
GPU	No dedicada	NVIDIA (CUDA)
Aceleración GPU	N/A	RAPIDS si compatible

El código base está diseñado para funcionar sin GPU. En la Máquina B se evaluará la posibilidad de usar RAPIDS Accelerator for Apache Spark. Si la versión de RAPIDS no es compatible con Spark 3.5.4, la comparación se hará solo con CPU pero aprovechando las diferencias de hardware (CPU, RAM, disco).

6. Resultados Preliminares

Se ejecutó una corrida de prueba con aproximadamente 6 minutos de acumulación para validar la funcionalidad completa del pipeline. Los resultados a continuación son preliminares; la corrida final utilizará 20 minutos de acumulación para obtener una muestra más robusta.

6.1 Datos acumulados

Métrica	Valor
Tiempo de acumulación	~6 minutos
Ventanas generadas	430
Símbolos monitoreados	8 (BTC, ETH, SOL, BNB, XRP, DOGE, ADA, AVAX)
Ventanas por símbolo	~54-63
Tipo de ventana	Sliding (1 min duración, 30s slide)

6.2 Resultados de K-Means (Fase 2)

Métrica	Valor
Mejor k	3
Silhouette Score	0.8245
Anomalías detectadas	24 / 430 (5.6%)
Umbral de anomalía (p95)	2.2086

El Silhouette Score de 0.82 indica una separación muy buena entre los clusters. Los 3 clusters representan estados de mercado bien diferenciados. El 5.6% de ventanas etiquetadas como anómalas es consistente con el umbral del percentil 95.

6.3 Resultados de Logistic Regression (Fase 3)

Métrica	Valor
Accuracy	94.57%
AUC-ROC	1.0000
F1-score (weighted)	91.92%
Split train/test	338 / 92
Anomalías en train	19
Anomalías en test	5

Coefficientes del modelo

Feature	Coefficiente	Interpretación
price_speed	0.53	Más influyente - rango amplio de precios
volatility	0.43	Alta fluctuación relativa
pct_return	Bajo	Menor impacto en clasificación
volume_intensity	Bajo	Menor impacto en clasificación

Los coeficientes confirman que price_speed y volatility son los features más relevantes para la detección de anomalías, lo cual es coherente con la intuición de que cambios rápidos y amplios en el precio caracterizan comportamiento anómalo.

6.4 Resultados de Inferencia (Fase 4)

Métrica	Valor
Ventanas clasificadas	254
Predicciones 'normal'	232 (91.3%)
Predicciones 'anómalo'	22 (8.7%)

6.5 Observaciones y limitaciones de la prueba

Desbalance de clases: Con solo 5 anomalías en el set de test, el modelo tiende a predecir 'normal' por defecto con el umbral de 0.5. Sin embargo, el AUC-ROC perfecto (1.0) indica que el ranking de probabilidades es correcto - el modelo sí distingue las anomalías, pero el umbral de decisión necesita ajustarse para la tasa de anomalías real (~5%).

Con más datos (corrida final de 20 minutos), se espera: mayor número de anomalías en ambos sets (train y test), mejor aprendizaje de la frontera de decisión, y métricas más representativas del rendimiento real del modelo.

7. Estructura del Proyecto

```

proyecto_final/
+-- config/spark_config.py      # Configuración centralizada de Spark
+-- hardware_info.py           # Fase 0: specs de hardware
+-- verify_environment.py       # Fase 0: verificación Spark-Kafka
+-- producer/binance_producer.py # Fase 1: WebSocket -> Kafka
+-- streaming/spark_streaming.py # Fase 2: estadísticos + K-Means
+-- streaming/spark_inference.py # Fase 4: inferencia en streaming
+-- training/train_model.py     # Fase 3: Logistic Regression
+-- metrics/capture_metrics.py  # Fase 5: métricas de Spark UI
+-- visualization/prepare_tableau.py # Fase 6: datos para Tableau
+-- models/kmeans_model/       # Modelo K-Means guardado
+-- models/logistic_model/     # Modelo LR guardado
+-- output/statistics/         # Estadísticos por ventana
+-- output/labeled_data/       # Datos etiquetados (K-Means)
+-- output/predictions/        # Predicciones (Fase 4)
+-- output/metrics/            # Métricas de Spark UI

```

8. Instrucciones de Ejecución

El pipeline se ejecuta en orden estricto. Cada fase debe completarse antes de iniciar la siguiente.

```
# Fase 0 - Verificación del entorno
python hardware_info.py
python verify_environment.py

# Fase 1 - Productor (dejar corriendo en terminal separada)
python producer/binance_producer.py

# Fase 2 - Primer streaming (estadísticos + K-Means)
python streaming/spark_streaming.py

# Fase 3 - Entrenar modelo supervisado
python training/train_model.py

# Fase 4 - Segundo streaming (inferencia)
python streaming/spark_inference.py

# Fase 5 - Captura de métricas (mientras corre Fase 2 o 4)
python metrics/capture_metrics.py --arch mac-m1
```

9. Pasos Siguientes para la Entrega Final

- Configurar ACCUMULATION_SECONDS = 1200 (20 minutos) para la corrida final.
- Limpiar todos los outputs previos y ejecutar el pipeline completo de inicio a fin.
- Capturar métricas de Spark UI durante las fases de streaming.
- Ejecutar el mismo pipeline en la Máquina B (Linux + GPU).
- Comparar métricas cuantitativamente entre ambas arquitecturas.
- Construir dashboards en Tableau con los datos consolidados.
- Redactar el informe final con análisis comparativo completo.